مقدمة الفصل الثاني — عمليات البرمجيات (Software Processes)

التعريفات الأساسية

- **Software Process:** مجموعة من الأنشطة المترابطة التي تهدف في النهاية إلى إنتاج **نظام برمجي**.
- **Managed Activity:** البرمجة الاحترافية ليست مجرد "كود"، بل هي نشاط **مُدار** يتطلب تخطيطاً وتنظيماً.
- ⚠ **قاعدة MCQ:** اختيار العملية يعتمد على: (نوع البرنامج + متطلبات العميل + مهارات الفريق).

1 الأنشطة الأربعة الأساسية

توجد هذه الأنشطة في **كل** عملية برمجية، لكن الفرق يكمن في كيفية ترتيبها (متسلسلة أم متداخلة).

النشاط	الاسم (English)	الكلمة المفتاحية (Keyword)	السؤال الجوهري
التحديد	Specification	تحديد الوظائف والقيود	ماذا سنبني؟
التطوير	Development	التصميم والبناء	كيف سنبنيه؟
التحقق	Validation	مطابقة حاجة العميل	هل بنيناه صح؟
التطور	Evolution	التحديث والاستمرارية	كيف نعدله لاحقاً؟

2 عناصر وصف العملية البرمجية

عندما نصف عملية برمجية لشخص ما، نستخدم ثلاثة مفاهيم أساسية:

1. **المخرجات (Products / Deliverables):** النتائج الملموسة لكل نشاط (مثلاً: نموذج المعمارية هو مخرج لنشاط التصميم).
2. **الأدوار (Roles):** توزيع المسؤوليات (مثلاً: مبرمج، مدير مشروع، مختص ضبط الجودة).
3. **الشروط (Conditions):**
   - **Pre-condition:** شرط يجب توفره **قبل** البدء (مثلاً: موافقة العميل قبل التصميم).
   - **Post-condition:** شرط يجب تحقيقه **بعد** الانتهاء (مثلاً: مراجعة الكود قبل التسليم).

3 التخطيط مقابل الرشاقة (Plan-Driven vs. Agile)

هذا هو التقسيم الأهم في هندسة البرمجيات المعاصرة.

المعيار	المنقادة بالتخطيط (Plan-Driven)	العمليات الرشيقة (Agile)
فلسفة التخطيط	كل شيء مخطط مسبقاً (UP-front).	تخطيط تدريجي ومستمر (Incremental).
قياس التقدم	مقارنة الإنجاز بالجدول الزمني.	بناءً على البرنامج العامل فعلياً.
المرونة	صعبة ومكلفة.	سهلة وجزء من العملية.
الأنسب لـ	**الأنظمة الحرجة والضخمة.**	**أنظمة الأعمال المتغيرة.**

4 كيف تختار العملية المناسبة؟ (توقعات MCQ)

في الامتحان، قد يذكر لك نوع النظام ويسألك عن العملية الأنسب:

- **الأنظمة الحرجة (Safety-Critical):** تحتاج لنمط **Plan-Driven** لأنها تتطلب توثيقاً دقيقاً وسجلات أمان.
- **أنظمة الأعمال (Business Systems):** تحتاج لنمط **Agile** لتستجيب لتقلبات السوق بسرعة.
- **الأنظمة الكبرى (Large Systems):** تحتاج إلى **مزيج (Hybrid)** يجمع بين التخطيط والمرونة.

كبسولات MCQ (خلاصة الخلاصة)

- **التعريف:** الـ Software Process هي أنشطة مترابطة لإنتاج نظام.
- **العالمية:** لا توجد عملية برمجية عالمية (Universal Process).
- **Deliverables:** هي النتائج التي تخرج من الأنشطة.
- **Pre-condition:** هو ما يجب أن يحدث "أولاً" قبل النشاط.
- **الأنظمة الحرجة:** دائماً ترتبط بالتوثيق والتخطيط المسبق (Plan-Driven).

ملخص القسم 2.1 — نماذج عمليات البرمجيات (SDLC Models)

معدّ خصيصاً لمراجعة امتحانات الـ MCQ

مفاهيم تأسيسية

- **Software Process:** مجموعة أنشطة مترابطة تؤدي إلى إنتاج نظام برمجي.
- **Software Process Model:** تمثيل مبسّط وعملي لعملية البرمجيات.
- ⚠ **قاعدة ذهبية:** لا يوجد نموذج "عالمي" واحد؛ اختيار النموذج يعتمد على **نوع المشروع** وظروفه.

⚙ الأنشطة الأربعة الأساسية (موجودة في أي نموذج)

النشاط	المعنى (Definition)
Specification	تحديد وظائف النظام والقيود المفروضة عليه.
Development	مرحلة التصميم وبناء الكود (Implementation).
Validation	التحقق من أن البرنامج يلبّي متطلبات العميل (التأكد من الجودة).
Evolution	تطوير وتحسين البرنامج لمواكبة التغييرات المستقبلية.

1 نموذج الشلال (Waterfall Model)

هو أول نموذج تم نشره (Royce 1970)، وهو نموذج **خطي تسلسلي** يعتمد على التخطيط المسبق (**Plan-Driven**).

← المراحل بالترتيب:

1. **Requirements Definition:** تحديد المتطلبات.
2. **Software Design & System:** تصميم النظام.
3. **Unit Testing & Implementation:** البرمجة واختبار الأجزاء الصغيرة.
4. **System Testing & Integration:** دمج الأجزاء واختبار النظام كاملاً.
5. **Maintenance & Operation:** التشغيل والصيانة (**أطول مرحلة**).

- ✔ **أفضل استخدام:** الأنظمة المدمجة (Embedded)، الأنظمة الحرجة (Critical Systems)، المشاريع الضخمة متعددة الشركات.
- ✕ **العيوب:** صعوبة التغيير، الجمود، احتمالية إنتاج نظام لا يلبّي حاجة المستخدم بسبب "تجميد المتطلبات" مبكراً.

2 التطوير التزايدي (Incremental Development)

هو الأسلوب **الأكثر شيوعاً** اليوم. يعتمد على تسليم النظام كـ "دفعات" أو زيادات متتالية.

- **الفكرة:** الأنشطة (Specification, Development, Validation) متداخلة وليست منفصلة.
- ✅ **المزايا (هام جداً للـ MCQ):**
  - تقليل تكلفة التغيير (وثائق أقل لإعادة الكتابة).
  - الحصول على تغذية راجعة (Feedback) مبكرة من العميل.
  - تسليم سريع لجزء من النظام (وإن لم يكتمل كلياً).
- ✕ **المشكلات:** صعوبة قياس التقدم للمدراء، تدهور بنية الكود (Architectural Corruption) مع كثرة الإضافات.
- 💡 **الحل:** استخدام الـ **Refactoring** (إعادة هيكلة الكود) لتحسين جودته دون تغيير وظيفته.

---

3 التكامل والتهيئة (Integration and Configuration)

يعتمد على فلسفة **إعادة الاستخدام (Reuse)** بدلاً من البرمجة من الصفر.

- **أنواع المكونات المستخدمة:**
  1. أنظمة جاهزة (Stand-alone apps).
  2. حزم برمجية (Objects/Packages) مثل Java Spring.
  3. خدمات الويب (Web Services).
- ✅ **المميزات:** تقليل التكلفة والمخاطر، سرعة التسليم.
- ✕ **العيوب:** تقديم تنازلات في المتطلبات، فقدان السيطرة على تطور النظام المستقبلي.

☆ عملية راشونال الموحدة (RUP)

نموذج شامل من شركة Rational (IBM حالياً)، يجمع بين مزايا النماذج السابقة.

المرحلة	الهدف الأساسي
Inception	وضع الجدوى التجارية وتحديد نطاق المشروع.
Elaboration	فهم المتطلبات وتطوير معمارية النظام (Architecture).
Construction	تنفيذ البرنامج (البرمجة والتصميم).
Transition	تسليم النظام للمستخدمين ونشره.

جدول المقارنة الشامل (مفتاح الـ MCQ)

المعيار	Waterfall	Incremental	Config & Integration
النوع	Plan-driven	Plan / Agile	Reuse-based
التسليم	دفعة واحدة في النهاية	دفعات متعددة متتالية	غالباً دفعة واحدة
التغيير	صعب ومكلف جداً	سهل ومرن	محدود بما توفره المكونات
التوثيق	كثيف (Document-heavy)	أقل	متوسط
الأنسب لـ	الأنظمة الحرجة والضخمة	معظم تطبيقات الويب والجوال	أنظمة الأعمال السريعة
العيب الأكبر	الجمود وعدم المرونة	تدهور الكود مع الوقت	التنازل عن بعض المتطلبات

كبسولات MCQ

- **Waterfall:** مستوحى من الهندسة العسكرية + أول نموذج منشور.
- **أطول مرحلة:** هي الصيانة (Maintenance) في نموذج الشلال.
- **Incremental:** هو الأسلوب "المسيطر" حالياً في تطوير البرمجيات.
- **Refactoring:** هو الحل لمشكلة تدهور الكود في التطوير التزايدي.
- **Reuse-based:** هو الاسم الآخر لنموذج (Configuration & Integration).
- **الأنظمة الحرجة (Safety-critical):** دائماً تفضل نموذج الـ **Waterfall**.

ملخص القسم 2.2 — أنشطة العملية (Process Activities)

نظرة عامة

العمليات البرمجية ليست خطوطاً جامدة، بل هي **أنشطة متداخلة** (تقنية، إدارية، وتعاونية).

- **في نموذج الشلال (Waterfall):** الأنشطة متسلسلة (واحدة تلو الأخرى).
- **في النموذج التزايدي (Incremental):** الأنشطة متداخلة (Interleaved).

1 تحديد المتطلبات (Software Specification)

الاسم المرادف: Requirements Engineering.

تعتبر **أحرج وأخطر مرحلة**؛ لأن الخطأ هنا يكلف الكثير لاحقاً.

- **Feasibility Study (دراسة الجدوى):** هي أول خطوة (قصيرة وغير مكلفة) للإجابة على سؤال: هل نكمل المشروع أم لا؟

مراحل هندسة المتطلبات الثلاث:

1. **Analysis & Elicitation:** جمع المتطلبات من العميل (مقابلات، ملاحظة).
2. **Specification:** كتابة المتطلبات في وثيقة رسمية.
3. **Validation:** التأكد أن المتطلبات واقعية ومنطقية.

فرق MCQ:

- **User Requirements: وصف عام للعميل.**
- **System Requirements:** وصف تقني مفصل للمطورين.

2 التصميم والتنفيذ (Implementation & Design)

تحويل المتطلبات "النظرية" إلى نظام "عملي" قابل للتشغيل.

أنشطة التصميم الأربعة (بالترتيب المنطقي):

1. **Architectural Design:** تصميم البنية العامة.
2. **Database Design:** تصميم هياكل البيانات.
3. **Interface Design:** تحديد كيف تتخاطب المكونات مع بعضها.
4. **Component Selection:** اختيار أو تصميم المكونات التفصيلية.

فرق جوهري: Testing vs. Debugging

الاختبار (Testing)	تصحيح الأخطاء (Debugging)
إثبات **وجود** أخطاء (نقد النظام).	**تحديد مكان** الخطأ وإصلاحه.
نشاط "تحقق".	نشاط "برمجي/تطويري".

3 التحقق من النظام (Software Validation)

الاسم المرادف: (V&V) Validation & Verification.
الهدف هو التأكد أن النظام يطابق المواصفات ويرضي العميل.

مراحل الاختبار (يجب حفظ الترتيب):

1. **Component Testing:** يجريه المطور على الأجزاء المنفردة.
2. **System Testing:** يجريه فريق مستقل للتأكد من تفاعل الأجزاء معاً.
3. **Customer Testing:** يجريه العميل ببيانات حقيقية (آخر خطوة قبل القبول).

Beta Testing: نوع من اختبار العميل للمنتجات التي ستباع في السوق (Commercial products).

نموذج V-Model
يربط كل مرحلة تطوير بمرحلة اختبار مقابلة لها. يستخدم غالباً في الأنظمة الحرجة (Plan-Driven).

---

4 تطور البرمجيات (Software Evolution)

تغيير النظام بعد تسليمه لتلبية احتياجات جديدة.

- **القاعدة:** البرمجيات مرنة، وتغييرها أرخص بكثير من تغيير الأجهزة (Hardware).
- **الفكرة الجوهرية:** التطوير والصيانة (Maintenance) هما **استمرارية واحدة (Continuum)** وليسا مرحلتين منفصلتين.

جدول المقارنة السريع (للمراجعة قبل اللجنة)

النشاط	الكلمة المفتاحية (Keyword)	أهم مخرج (Output)
Specification	Requirements Engineering	Requirements Document
Design	Architectural / Database	Software Architecture
Validation	V / Testing & V	Ready-to-use System
Evolution	Continuum / Change	Updated System

نقاط MCQ ذهبية (لا تنساها!)

- **أخطر مرحلة؟** Specification.
- **Testing ≠ Debugging؟** نعم، الاختبار يجد الخطأ، والديباجينج يصلحه.
- **ترتيب الاختبار؟** Customer ← System ← Component.
- **دراسة الجدوى (Feasibility)؟** تتم **قبل** البدء الفعلي بجمع المتطلبات.
- **الفرق في Agile؟** المتطلبات لا تُكتب في وثائق ضخمة، بل تُحدد قبل كل Increment.
- **في Waterfall؟** الأنشطة منفصلة تماماً وموقعة (Signed-off).

ملخص القسم 2.3 — التعامل مع التغيير (Coping with Change)

استراتيجيات التعامل مع التغيير

هناك فلسفتان للتعامل مع متطلبات التغيير، ولكل منهما أداتها الخاصة:

الاستراتيجية	المعنى	الأداة الرئيسية	الهدف
Change Anticipation	**توقع** التغيير قبل حدوثه.	Prototyping	تقليل التعديلات الكبيرة لاحقاً.
Change Tolerance	**استيعاب** التغيير وتسهيله.	Incremental Delivery	جعل التغيير أرخص وأسرع.

1 النمذجة الأولية (Software Prototyping)

هي نسخة سريعة "تُرمى" لاحقاً (**Throw-away**)، تهدف لاستكشاف المتطلبات وتجربة الواجهات.

مراحل تطوير النموذج (4 خطوات):

1. **Objectives:** تحديد الهدف (لماذا نبني النموذج؟).
2. **Functionality:** تحديد ما سيتم تضمينه (وما سيتم حذفه).
3. **Development:** البناء السريع.
4. **Evaluation:** التقييم مع المستخدمين.

ما الذي يتم حذفه في الـ Prototype؟ (نقطة MCQ هامة)
لتوفير الوقت، نتجاهل:

- المتطلبات غير الوظيفية (مثل السرعة واستهلاك الذاكرة).
- معالجة الأخطاء (Error Handling).
- معايير الموثوقية (Reliability).

2 التسليم التزايدي (Incremental Delivery)

هنا نحن لا نطور فقط على دفعات، بل **نسلم** دفعات فعلية تعمل في بيئة العميل.

الفرق الجوهري :

- **Incremental Development:** نطور النظام على أجزاء (داخلياً)، وقد لا يرى العميل شيئاً حتى النهاية.
- **Incremental Delivery:** العميل يستلم "جزءاً يعمل" ويبدأ باستخدامه فعلياً في عمله.

المزايا والعيوب:

- **المزايا:** قيمة مبكرة للعميل، تغذية راجعة حقيقية، اختبار مكثف للخدمات المهمة (لأنها تسلم أولاً).
- **العيوب:** صعوبة مع الأنظمة القديمة (Legacy)، تعارض مع العقود التقليدية، صعوبة تحديد "الخدمات المشتركة".

متى نتجنب التسليم التزايدي؟ في الأنظمة **الحرجة** (Safety-Critical)، والأنظمة **المدمجة** (Embedded)، والمشاريع **الضخمة** الموزعة.

المقارنة النهائية: Prototype vs. Incremental Delivery

المعيار	Prototyping	Incremental Delivery
المصير	يُرمى (Throw-away)	يستمر ويتطور كمنتج نهائي
نوع الاستجابة	Anticipation (توقع)	Tolerance (تحمل/استيعاب)
البيئة	تجريبية / مخبرية	بيئة العمل الحقيقية
التكلفة	منخفضة جداً	تكلفة تطوير كاملة

كبسولات MCQ (خلاصة القسم 2.3)

- **Rework:** هي التكلفة الإضافية التي تظهر بسبب التغيير.
- **الهدف من Prototype:** هو التحقق من المتطلبات (**Validation**) وليس الإنتاج.
- **الأولوية في Incremental Delivery:** الخدمات ذات القيمة الأعلى والخدمات الأكثر عرضة للفشل تُسلم **أولاً**.
- **الـ Prototype في الـ MCQ:** إذا رأيت كلمة "Throw-away" أو "Explore requirements" فالإجابة Prototyping.
- **Refactoring:** هي عملية تحسين الكود داخلياً لجعل التغيير أسهل (Change Tolerance).

ملخص القسم 2.4 — تحسين العملية (Process Improvement)

المفهوم الأساسي

- **التعريف:** فهم العمليات الحالية وتغييرها بهدف رفع جودة المنتج، تقليل التكاليف، أو تسريع التطوير.
- **الهدف:** تلبية الطلب الصناعي المستمر على برمجيات **أرخص**، **أفضل**، وتُسلَّم في **مواعيد أضيق**.

أسلوبان لتحسين العملية

المعيار	مدخل نضج العملية (Process Maturity)	المدخل الرشيق (Agile Approach)
التركيز	تحسين الإدارة وإدخال الممارسات الجيدة.	تطوير تكراري وتقليل الأعمال الإضافية (Overhead).
الهدف الأساسي	جودة المنتج + قابلية التنبؤ.	تسليم سريع + استجابة للتغيير.
الأساس	**Plan-Driven** (يزيد الـ Overhead).	يقلل الإجراءات الرسمية والتوثيق.
المناسبة	الشركات الكبيرة.	الفرق الصغيرة والمتوسطة.

ملاحظة: لا يوجد أسلوب مثالي يناسب الجميع؛ فكل فريق غالباً ما ينحاز لأسلوبه الخاص.

---

1 دورة تحسين العملية (The Improvement Cycle)

خاصة بمدخل "نضج العملية"، وهي دورة مستمرة تتكرر لضمان التطور:

المراحل الثلاثة:

1. **Process Measurement (القياس):** قياس خصائص العملية لإنشاء **baseline** (خط أساس للمقارنة).
2. **Process Analysis (التحليل):** تقييم العملية وتحديد نقاط الضعف والعوائق (**bottlenecks**).
3. **Process Change (التغيير):** اقتراح وتطبيق تغييرات لمعالجة نقاط الضعف.

نقطة جوهرية: بدون بيانات قياس ملموسة، لا يمكن تقييم فائدة التحسين. التحسين نشاط **طويل الأمد** ومستمر.

---

2 نموذج نضج القدرة (CMM / Capability Maturity Model)

- **المؤسس:** طوّره معهد هندسة البرمجيات (**SEI**) عام 1988 بمقترح من **Humphrey**.
- **الهدف:** تقييم قدرات مقاولي البرمجيات لوزارة الدفاع الأمريكية.

المستويات الخمسة (بالترتيب من الأدنى للأعلى):

المستوى	الاسم	الخصائص الرئيسية
Level 1	**Initial**	أهداف العملية محددة، ونطاق العمل مُعلن للفريق.
Level 2	**Managed**	سياسات تنظيمية، خطط موثقة، إدارة موارد وإجراءات مراقبة.
Level 3	**Defined**	**توحيد ونشر العمليات** على مستوى المنظمة بالكامل.
Level 4	**Quantitatively Managed**	**تحكم إحصائي وكمّي** في العمليات الفرعية والقياسات.
Level 5	**Optimizing**	تحليل الاتجاهات واستخدام القياسات لدفع **التحسين المستمر**.

المقارنة النهائية للتحسين

المعيار	Process Maturity	Agile
الأساس	Plan-Driven	Iterative
التوثيق	كثير (Overhead عالٍ)	أقل ما يمكن
التركيز	العملية والإدارة	الكود والتسليم
الاستخدام	الشركات الكبيرة	الشركات الصغيرة والمتوسطة

نقاط MCQ احفظها جيداً

- ◈ **دورة التحسين:** Change ← Analyze ← Measure.
- ◈ **بدون قياس:** لا يوجد تحسين حقيقي يمكن تقييمه.
- ◈ **CMM:** طوّره SEI عام 1988 (Humphrey).
- ◈ **Level 1:** هو Initial (الأدنى) | **Level 5:** هو Optimizing (الأعلى).
- ◈ **Level 3 (Defined):** يركز على "توحيد العمليات" في كل المنظمة.
- ◈ **Level 4:** يركز على "التحكم الإحصائي" (Statistical Control).
- ◈ **الشركات الكبيرة:** هي الوحيدة التي تستخدم Maturity Approach حالياً لتكلفته العالية.